CIDADES ESG INTELIGENTES

Atividade avaliativa — Navegando pelo mundo DevOps

API ASP.NET Core .NET 8 containerizada, com SQL Server, CI/CD no GitHub Actions, imagem pública no GHCR e deploy automatizado em staging e produção.

INTEGRANTES

Guilherme Luz Martins — RM 564344

Henrique Miranda Paulino — RM 565689

Nicolas Granatelli Sanches — RM 565755

Henry Gois Ribeiro — RM 564329

Matheus Pedrosa Mendes da Costa — RM 564328

✓ PIPELINE APROVADO	13/10/2026	5/5 TESTES

1. Visão geral e arquitetura

O projeto implementa uma API REST para monitoramento ESG de eficiência energética. O trabalho original foi adaptado para simular um ciclo de produção: compilação, testes, empacotamento, publicação e deploy com validação de saúde.

Git push	GitHub Actions	GHCR	Staging	Produção
Código em main	Build + xUnit	Imagem por SHA	API + SQL	API + SQL

Componentes

- **API:** C# / ASP.NET Core .NET 8, Entity Framework Core e autenticação HMAC Bearer.
- **Banco:** SQL Server 2022, com migrations aplicadas automaticamente na inicialização.
- **Testes:** xUnit; cinco testes existentes executados localmente e no pipeline.
- **Observabilidade:** endpoint `/health` verifica também a disponibilidade do banco.
- **Entrega:** Dockerfile multi-stage, Compose local e Compose de deploy com rede, volume e variáveis de ambiente.

Resultado comprovado

Validação	Resultado
Restore / build Release	Aprovado — 0 erros e 0 avisos
Testes xUnit	Aprovado — 5 de 5
Imagem Docker	Publicada no GHCR com tags SHA e latest
Staging / Produção	Aprovados — /health HTTP 200

2. Pipeline CI/CD

Ferramenta: **GitHub Actions**. O workflow `.github/workflows/ci-cd.yml` é acionado em pull requests e pushes para `main`. O deploy usa exatamente a imagem imutável identificada pelo SHA do commit.

Etapa	Job	Lógica
1	**Build e testes**	Valida os arquivos Compose, restaura dependências, compila em Release e executa xUnit. O TRX é publicado como artefato.
2	**Imagem Docker**	Docker Buildx cria a imagem multi-stage e publica no GHCR com a tag do SHA e a tag latest.
3	**Staging**	Baixa a imagem, gera segredos efêmeros mascarados, sobe API + SQL Server e exige prontidão em /health.
4	**Produção**	Inicia somente depois do sucesso de staging; sobe ambiente isolado com a mesma imagem e repete o health check.
5	**Evidências**	Publica resultados de testes, logs e estado dos containers como artefatos; encerra os ambientes efêmeros.

Execução validada

- Run **#2** — status **Success** — duração total de **3m42s**.
- Commit: `608c0ff98b957e5cabcfc00107c6951bb8c40537`.
- Build/testes: 36s; imagem: 1m01s; staging: 55s; produção: 58s.
- Artefatos: test-results, staging-runtime-evidence e production-runtime-evidence.

Link verificável

github.com/henry-gois/cidadeEsgInteligente/actions/runs/38491269652

3. Containerização e orquestração

Dockerfile

- Build multi-stage com `mcr.microsoft.com/dotnet/sdk:8.0` e runtime `aspnet:8.0`.
- Restore e publish em Release; somente os binários publicados seguem para a imagem final.
- Execução como usuário não-root da imagem .NET; porta 8080; curl instalado para health checks.

Docker Compose

Serviço	Responsabilidade	Proteção / saúde
api	ASP.NET Core; migrations; endpoint /health	Health check HTTP; depende do banco healthy; segredos por environment
db	SQL Server 2022 Developer	Health check sqlcmd; senha por environment; volume persistente
esg-network	Rede bridge dedicada	Comunicação interna pelo nome db
esg_sqlserver_data	Volume nomeado	Preserva dados entre recriações locais

Comandos principais

```
docker compose up --build -d --wait --wait-timeout 180

curl --fail http://localhost:8080/health

docker compose ps

docker compose down
```

A imagem foi criada de verdade no runner Linux e publicada como `ghcr.io/henry-gois/cidadeesginteligente:608c0ff...`. O pacote público possui também a tag `latest`.

4. Evidências — pipeline e imagem

As imagens abaixo são capturas originais das páginas públicas do GitHub. O pipeline mostra todos os jobs principais verdes; os jobs remotos opcionais aparecem ignorados porque a demonstração usa runners efêmeros.

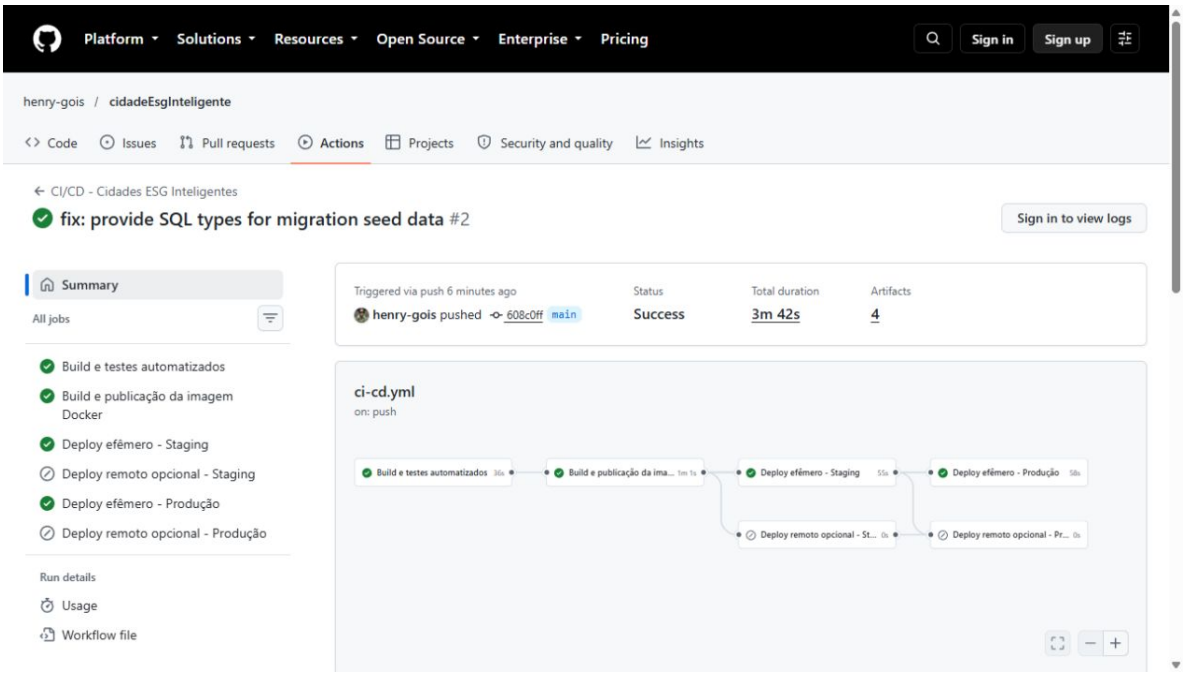


Figura 1 — Pipeline completo aprovado: build/testes, publicação Docker, staging e produção.

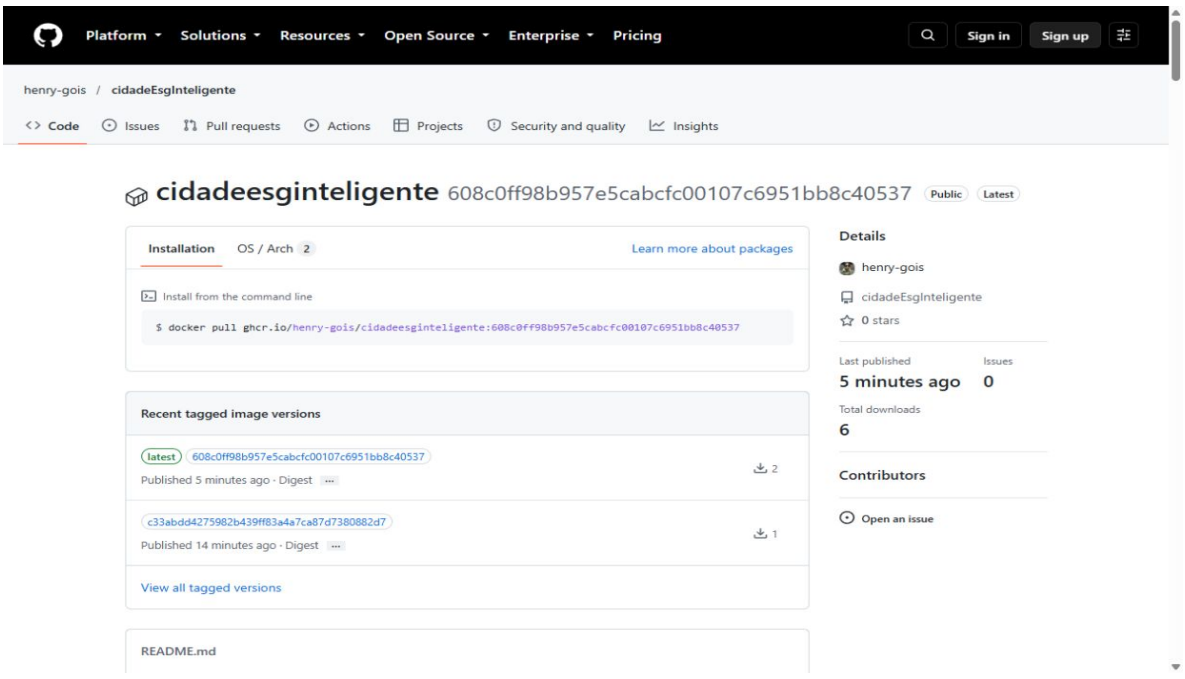


Figura 2 — Pacote público no GHCR com tags latest e SHA imutável.

5. Evidências — staging e produção

Cada ambiente subiu a mesma imagem imutável com um SQL Server isolado. A etapa de prontidão executa curl em `/health`, registra `HTTP 200 - {"status":"healthy"}` e interrompe o job em caso de falha.

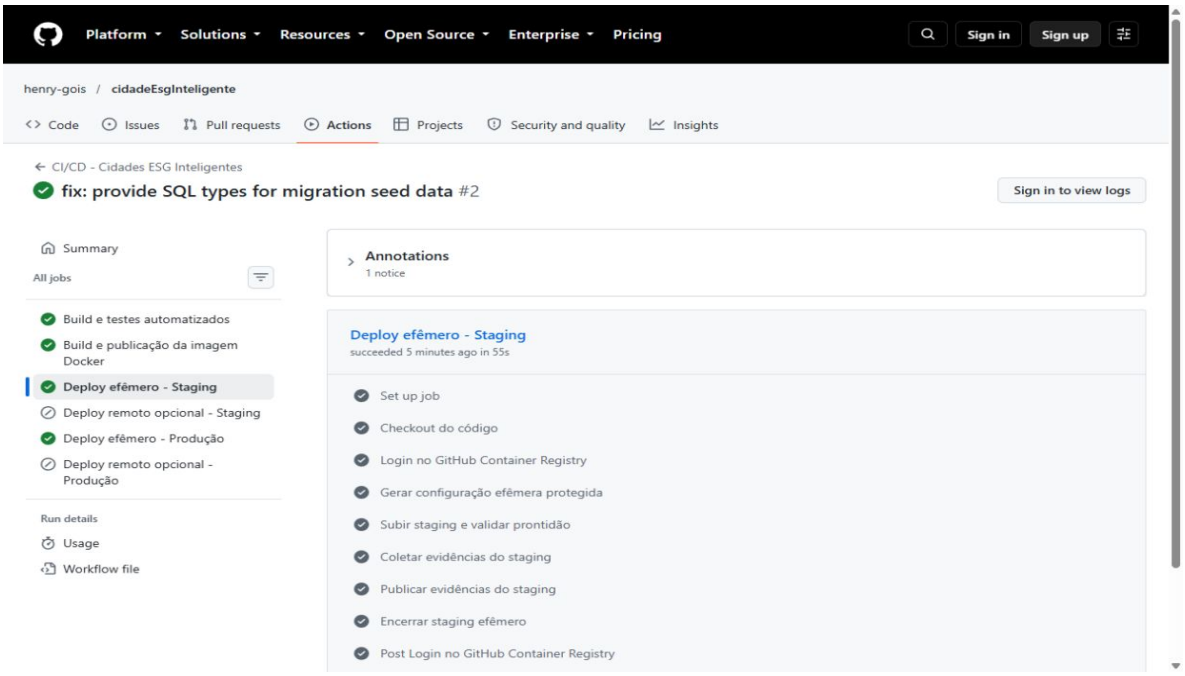


Figura 3 — Staging aprovado em 55s, incluindo subida e validação de prontidão.

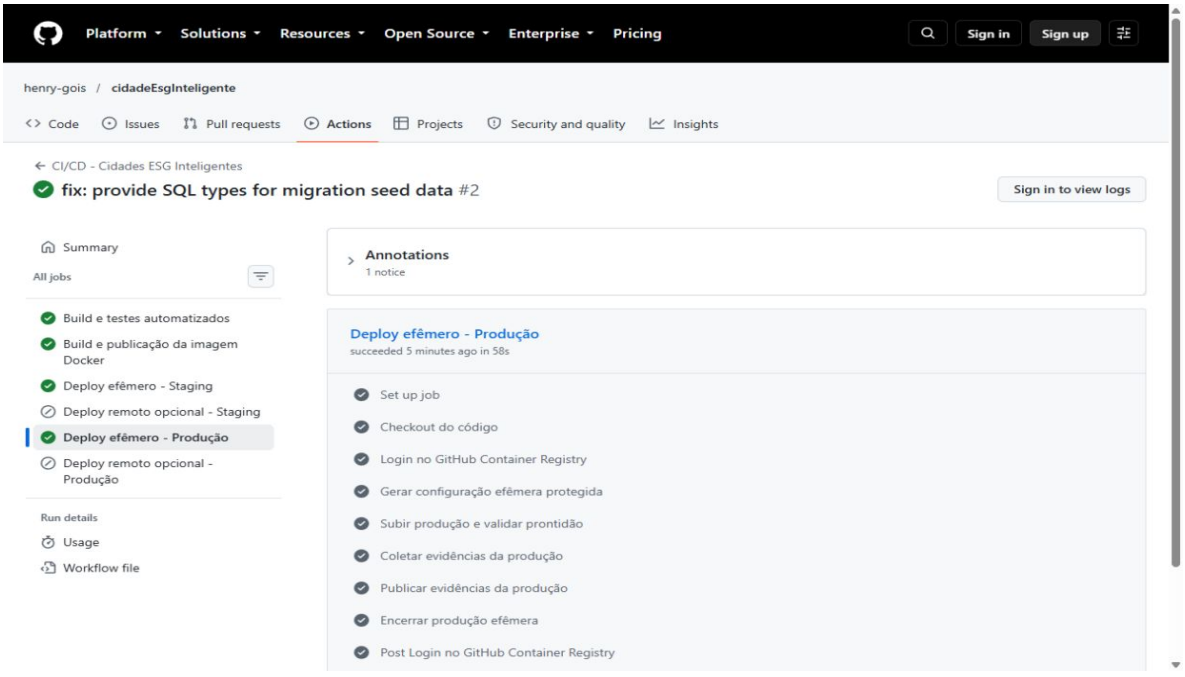


Figura 4 — Produção aprovada em 58s, iniciada somente após staging.

6. Desafios encontrados e soluções

Desafio	Como foi resolvido
Docker indisponível no computador local	A validação de containers foi movida para runners Linux do GitHub Actions. Assim, o build da imagem, o SQL Server e os dois deployments foram testados de forma reproduzível sem exigir Linux local.
Falha real na migration inicial	O primeiro deploy revelou que as operações InsertData não tinham metadados de tipo suficientes. Foram adicionados columnTypes explícitos; a execução seguinte concluiu as migrations e ficou verde.
Ausência de servidor próprio	Foram criados staging e produção efêmeros em runners separados, com GitHub Environments, projetos Compose e segredos distintos. O modo SSH persistente continua disponível como opção.
Segurança de credenciais	Senha SQL e chave HMAC são geradas aleatoriamente em cada job, mascaradas nos logs e descartadas ao final. Nenhum segredo real foi versionado.
Rastreabilidade	A imagem recebe a tag completa do SHA; produção usa a mesma tag aprovada em staging. Testes, logs e estado dos containers são preservados como artefatos.

Limite declarado

Os deployments padrão são reais, mas deixam de existir ao fim dos jobs. A comprovação permanece no histórico público, nos artefatos e nas capturas. Hospedagem pública permanente exige um provedor/servidor; o workflow SSH opcional já está preparado para essa extensão.

7. Checklist de entrega

Item	Status
Projeto compactado em ZIP com estrutura organizada	✓ OK
Dockerfile funcional	✓ OK
docker-compose.yml / orquestração	✓ OK
Pipeline com build, testes e deploy	✓ OK
README com instruções e prints	✓ OK
Documentação técnica com evidências	✓ OK
Imagem publicada no GHCR	✓ OK
Deploy realizado em staging	✓ OK
Deploy realizado em produção	✓ OK
Health check HTTP 200 nos dois ambientes	✓ OK

Referências verificáveis

- Repositório: github.com/henry-gois/cidadeEsgInteligente
- Pipeline: Actions run 38491269652
- Imagem: GHCR cidadeesginteligente

ENTREGA VALIDADA	Execução #2: SUCCESS	Commit 608c0ff